

Przeszukiwanie w grach

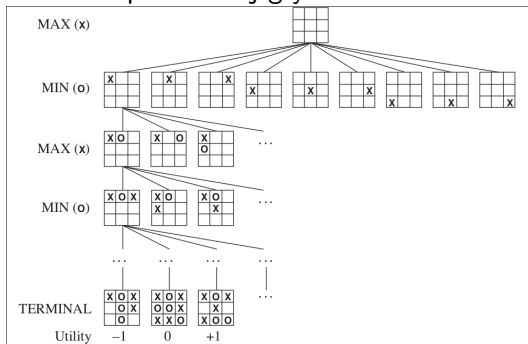
Paweł Rychlikowski

Instytut Informatyki UWr

15 kwietnia 2026

Drzewo gry. Powtórzenie

Fragment drzewa dla prawdziwej gry



Algorytm MinMax z głębokością. Powtórzenie

```
def decision(state):
    return max([a for actions(state),
                key = lambda a : minmax(result(a,state), MIN ,0)]])

def minmax(state, player, depth):
    if terminal(state): return utility(state)
    if cut_off_test(state, depth):
        return heuristic_value(state)

    values = [minmax(result(a,state), 1-player, depth+1) for a in actions(state)]
    if player == 0:
        return min(values)
    else:
        return max(values)
```

Dwa parametry algorytmu wyszukiwania

1. **cut_off_test**: kiedy kończymy przeszukiwanie
 - najłatwiej: jak osiągniemy maksymalny poziom, biorąc pod uwagę możliwości
 - Nie jest to jedyne wyjście (ani najlepsze)
2. Co to znaczy funkcja **heuristic_value**

Jak szacować wartość sytuacji?

Wariant 1

Korzystamy z wiedzy eksperta, próbując ją sformalizować.

Wariant 2

Próbujemy zaprząć jakiś mechanizm uczenia (lub przeszukiwania), żeby tę funkcję wybrać.

Jak szacować wartość sytuacji? (2)

Generalne wskazówki:

1. Przewaga materialna (więcej, lepszych figur)
2. Ustawienie figur (ruchliwość – liczba możliwych ruchów)
3. Szacowana liczba ruchów do zwycięstwa (zagrożony król, itp).
4. Ochrona naszych figur (jak mnie zbijesz, to ja cię zaraz zbiję)

Aktywny goniec

Biały goniec wprowadzony do gry, czarny nie może nic zrobić.



Przewaga materialna

- Wartość materialną liczą powszechnie szachiści:
 - a) pion: 1
 - b) skoczek, goniec: 3
 - c) wieża: 5
 - d) hetman: 9
- Sprawdzono doświadczalnie, że te wartości są dobrze dobrane (jak sobie wyobrazić taki eksperyment?)

Uwaga

Nawet nie wiedząc nic o uczeniu, możemy sobie wyobrazić łatwo jakąś procedurę wyznaczania tych wartości. Na przykład:

1. Losujemy 100 zestawów:
(1, wartość-gońca, wartość-skoczka, wartość-wieży, wartość-hetmana).
2. Przeprowadzamy pojedynki każdy z każdym.
3. Wybieramy zwycięzcę.

Connect 4. Przykładowa gra



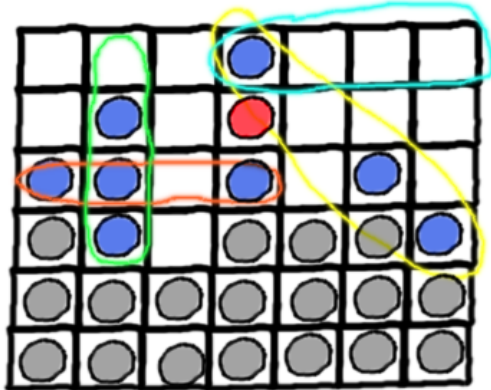
- Prosty, a zarazem grywalny wariant kółka i krzyżyka
- Dodatkowe elementy: mamy ciężenie i piony spadają, gramy do 4 w wierszu, kolumnie lub na przekątnej

Connect 4. Zwycięska konfiguracja



- Prosty, a zarazem grywalny wariant kółka i krzyżyka
- Dodatkowe elementy: mamy ciężenie i piony spadają, gramy do 4 w wierszu, kolumnie lub na przekątnej

Co to znaczy wzorzec w Connect 4?



- Analizujemy wszystkie czwórki pól (w każdej bowiem może się zdarzyć układ wygrywający)
- Czwórki, w których są pionki obu kolorów pomijamy
- Wyznaczamy wagę 1-ek, dwójek, trójek (być może zależnie od kierunków)

- Możliwe są większe wzorce, uwzględniające szerszy kontekst
- możliwe jest również **uczenie** większych wzorców. Na przykład za pomocą **splotowych sieci neuronowych (CNN)**.

Uwaga

Takie sieci działały w AlphaGo.

- Funkcja oceny może być ważoną sumą zaobserwowanych wzorców.
- Wzór:

$$\sum_i w_i p_i$$

(w_i – waga i-tego wzorca, p_i – ile razy ten wzorzec występuje na planszy)

- Niektóre wagi są dodatnie (mój dobry wzorzec, słabe ustawienie oponenta), inne ujemne.

Wyznaczanie wag funkcji oceniającej

- **Na oko** – heurystycznie dobieramy parametry, starając się wyeliminować niepożądane zachowania (proste, ale...)
- Algorytmy uczenia maszynowego (tak dobrać parametry, żeby funkcja oceniająca :
 - **przypominała** wartości przypisane sytuacjom przez ekspertów
 - **porządkowała** ruchy w ten sposób, by jak najczęściej najlepszy ruch wg niej był najlepszym ruchem wg eksperta
 - Ekspertem mogą być ludzcy gracze (a dokładniej historyczne decyzje prawdziwych graczy)
 - Szacowała prawdopodobieństwo wygranej (jak – regresja logistyczna, dyskusja na tablicy)
- Algorytmy ewolucyjne (następny slajd)

Drobna uwaga o ewolucji w grach: jak wyznaczyć parametry funkcji oceniającej?

- Pokusa, żeby zastosować algorytmy ewolucyjne jest naturalna (bo zadanie przypomina ewolucję, w której osobniki toczą ze sobą walkę).
- **Problem:** Jak wyznaczyć funkcję celu?
 - a) Rozgrywać turnieje, przystosowaniem jest średni wynik.
 - b) Wybrać grupę przeciwników (stałą), przystosowaniem X-a będzie średni wynik z tymi przeciwnikami.

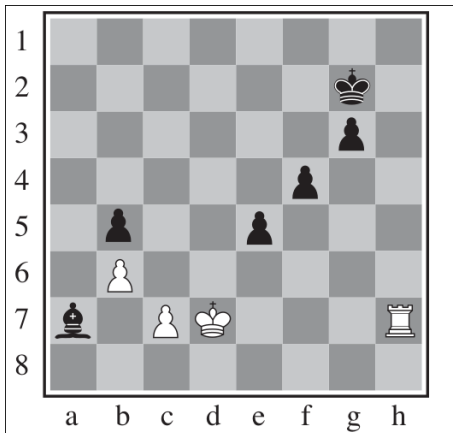
Uwaga

Opcja pełnej ewolucji trochę niebezpieczna, często łączy się oba warianty.

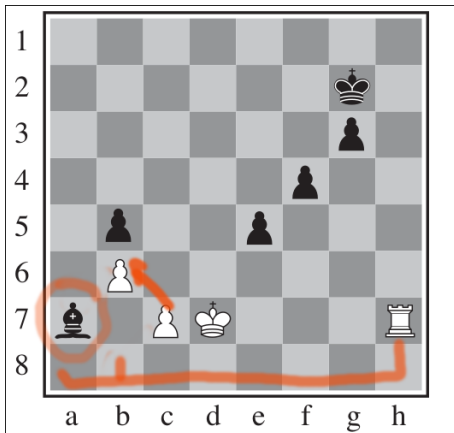
Drugi **metaparametr** funkcji obliczającej wartość planszy.

- Są dwa problemy związane z przerywaniem przeszukiwania:
 1. Przerwanie w niestabilnej sytuacji (na przykład w środku wymiany hetmanów)
 2. Tzw. efekt horyzontu (czyli widzimy, że coś się zdarzy, ale w odległej perspektywie)

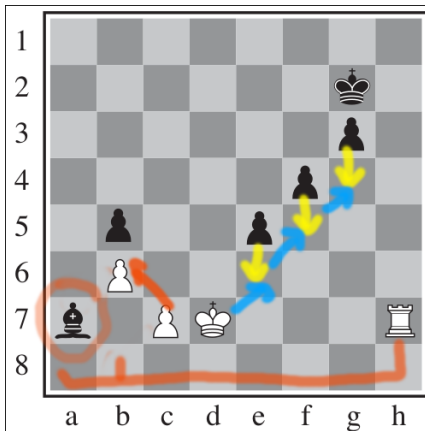
Efekt horyzontu (zła sytuacja czarnego gońca)



Efekt horyzontu (zła sytuacja czarnego gońca)



Efekt horyzontu (zła sytuacja czarnego gońca)



- Dwie groźby umieszczone jedna nad drugą (rysunek na tablicy)
- Strona przegrywająca może robić nieistotne ruchy w innych kolumnach (ale pod koniec będzie trzeba wykonać ruch w kluczowej kolumnie)

Kończenie przeszukiwań w praktyce

- Nieprzerywanie, jeżeli przeciwnik ma bicie.
- Ogólniej: powyżej jakiejś głębokości rozważamy tylko ruchy **mocno zmieniające sytuację**

Definicja

W **przeszukiwaniu z bezruchem** (**quiescence search**) możemy skończyć poszukiwanie **tylko** gdy sytuacja jest statyczna.

Kończenie przeszukiwań w praktyce

- Można też stosować jakąś wersję *local beam search* (od któregoś momentu ograniczając mocno rozgałęzienie drzewa)
- Rozważa się warunek **singular extension**, czyli istnienie jednego ruchu, który jest wyraźnie (na oko) lepszy od innych. Takie ruchy zawsze wykonujemy, zwiększając głębokość, a nie zwiększając rozgałęzienia.

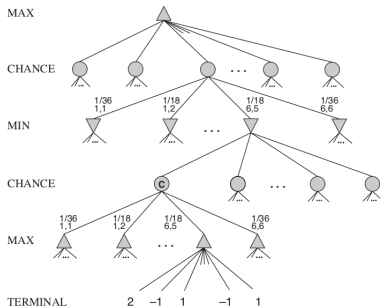
Uwaga

Trochę tak działają ludzie.

- W niektórych grach (i w życiu) mamy element losowy.
- Prosty przykład: [szachy z kostką](#):
 - Przed ruchem wykonujemy rzut kostką, który determinuje czym możemy się ruszyć,
 - 1 - pionek, 2 - skoczek, 3 - goniec, 4 – wieża, 5 – hetman, 6 – król
 - Gramy do zbitcia króla.

Losowość w grach

- Wprowadzamy dodatkowe węzły, czyli **chance nodes**.
- Przykładowe drzewo gry (dla losowania przy użyciu **dwóch kości**):



- Minimax, do którego dołożono węzły losowe.
- W węzłach losowych mamy wybór wartości oczekiwanej (sumowanie)

```
def emm(state, player):  
    if terminal(state): return utility(state)  
    if player == MIN:  
        return min( emm(result(state, a), next(player)) for a in actions(state))  
    if player == MAX:  
        return max( emm(result(state, a), next(player)) for a in actions(state))  
    if player == CHANCE:  
        return sum( P(r) * emm(result(state, r), next(player)) for r in actions(state))
```

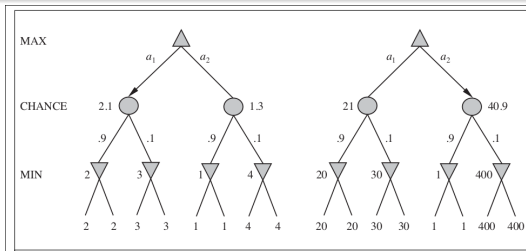
Wartość sytuacji w grach z losowością

Uwaga 1

Dowolne monotoniczne przekształcenie nie zmienia ruchów wybieranych przez minimax!

Uwaga 2

W grach z losowością powyższe zdanie przestaje być prawdziwe.



- Analiza gier z losowością jest nieco trudniejsza.
- Możemy skorzystać z następującej idei:
Oceniamy sytuację przeprowadzając dużo losowych gier rozpoczynających się w danej sytuacji
- **Uwaga:** dwa rodzaje losowości: jeden związany z węzłami losowymi (dany przez grę), drugi związany z węzłami min/max – zamiast wyliczać ruch wykonujemy ruch losowy.

Uwaga

Monte Carlo Simulation dotyczy nie tylko gier z losowością!

Monte Carlo Simulation

- Zauważmy, że Monte Carlo Simulation jakoś rozwiązuje problem horyzontu (bo symulacje mogą być b. długie)
- Możemy losować ruchy z niejednakowym prawdopodobieństwem (preferując te, które lokalnie wyglądają sensownie)

Uwaga

Bardzo ważnym nie tylko w grach jest algorytm **Monte Carlo Tree Search**, o którym jeszcze powiemy.

- Ciekawe do analizy są gry, w których agenci nie mają pełnej wiedzy o świecie.
- Klasyczne przykłady to gry karciane, ale nie tylko.

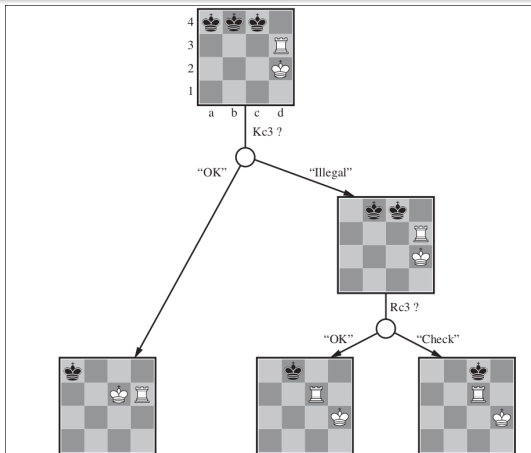
Kriegspiel

- Mamy dwóch graczy, arbitra i 3 szachownice.
- Gracze widzą na szachownicy swoje pionki, mogą tworzyć też hipotezy o bierkach przeciwnika.
- Arbiter zna położenie wszystkich figur i udziela graczom pewnych (skąpych) informacji.
 - a) przede wszystkim ocenia, czy ruch jest możliwy (komunikacja osobista, dobry ruch jest od razu wykonywany, w przypadku złego, gracz proponuje kolejny, aż do skutku)
 - b) odpowiada na pytanie: „czy ja (gracz) mam jakieś bicie?”
 - c) informuje obu graczy, że „na polu X zbito bierkę” (nie podając jaka bierka jest zbita, a jaka biła)
 - d) Mówi o szachu (do ubu graczy), dodając, że zagrożenie jest w wierszu, kolumnie, przekątnej lub przez skoczka
- Tak poza tym, to całkiem normalne szachy.

Podobno ludzie radzą sobie z tą grą całkiem nieźle...

Końcówka w Kriegspiel

Przykładowa końcówka, gracz biały dowiedział się, że czarnemu został tylko król i jest on na jednym z 3 pól.



Uwaga 1

W stanie gry powinniśmy umieścić możliwe ustawienia bierok przeciwnika

Trochę jak z komandosem...